



Get started with GitHub Copilot in VS Code

GitHub Copilot transforms how you write code in Visual Studio Code. In this hands-on tutorial, you build a complete task management web application while discovering VS Code's AI capabilities: intelligent inline suggestions, autonomous feature development with agents, precise editing with inline chat, integrated smart actions, and powerful customization options.

By the end of this tutorial, you'll have both a working web application and a personalized AI coding setup that adapts to your development style.

▶ Create the sample app

Use chat in VS Code to generate the sample application in one go.

Prerequisites

- VS Code installed on your machine. Download it from the [Visual Studio Code website](#).
- Access to GitHub Copilot. Follow these steps to [Set up GitHub Copilot in VS Code](#).

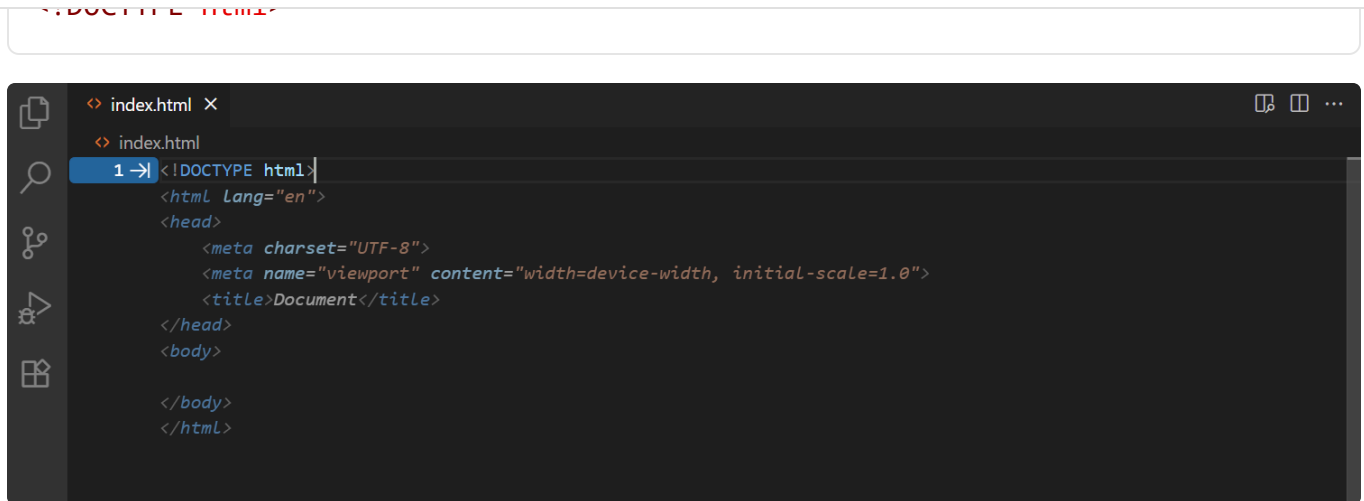
💡 Tip

If you don't have a Copilot subscription, you can sign up to use Copilot for free directly from within VS Code and get a monthly limit of inline suggestions and chat interactions.

Step 1: Experience inline suggestions

AI-powered inline suggestions appear as you type, helping you write code faster and with fewer errors. Let's start building the foundation of your task manager.

- 1 Create a new folder for your project and open it in VS Code.
- 2 Create a new file called `index.html`.
- 3 Start typing the following and, as you type, VS Code provides inline suggestions (*ghost text*):

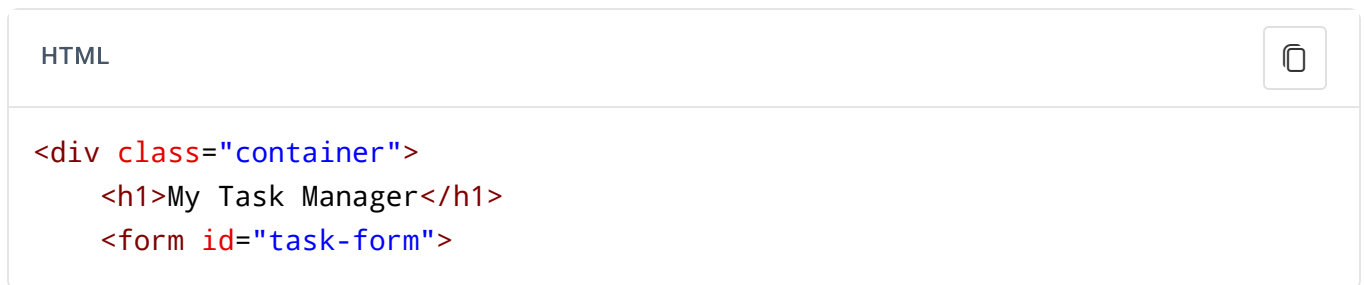


You might see different suggestions because large language models are nondeterministic.

- 4 Press **Tab** to accept the suggestion.

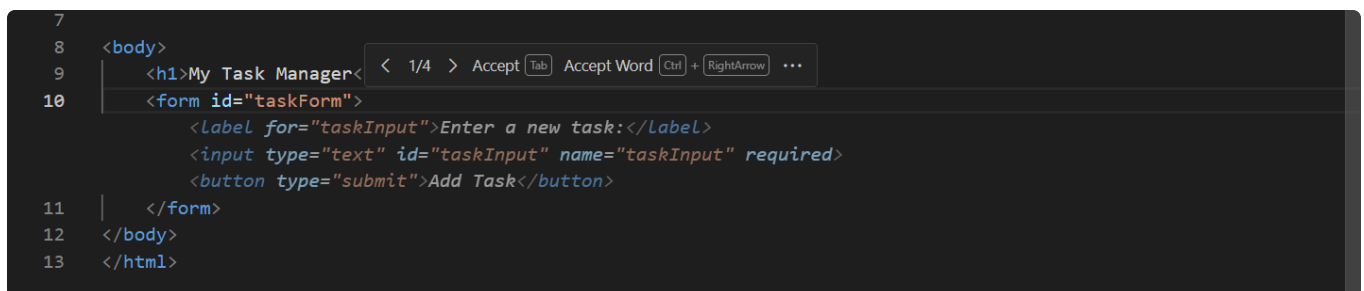
Congratulations! You've just accepted your first AI-powered inline suggestion.

- 5 Continue building your HTML structure. Inside the `<body>` tag, start typing:



Notice how VS Code continues suggesting relevant HTML elements as you build your application structure.

- 6 If you see multiple suggestions, hover over the ghost text to see navigation controls, or use **Alt+]** and **Alt+[** to cycle through options.



Inline suggestions work automatically as you type, learning from your patterns and the context of your project. They're particularly helpful for writing boilerplate code, HTML structures, and repetitive patterns.

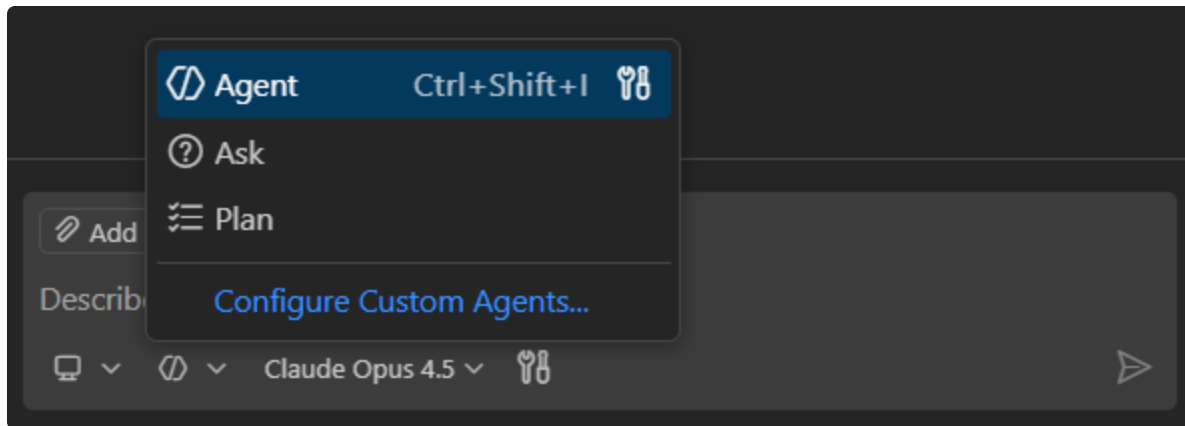
Step 2: Build complete features with agents

AI Agents are VS Code's most powerful AI capability. Given a natural language prompt, they autonomously plan and implement complex features across multiple files. Let's use them to create the core functionality of your task manager application.

- 1 Open the Chat view by pressing **Ctrl+Alt+I** or by selecting the chat icon in the VS Code title bar.

The Chat view is where you interact with the AI by using natural language prompts. You can have an ongoing conversation and iteratively refine your requests to get better results.

- 2 Select **Agent** in the agent dropdown menu to let the AI independently implement your request end-to-end.



🚨 Important

If you don't see the agent option, make sure agents are enabled in your VS Code settings ([chat.agent.enabled](#) ORG). Your organization might also have disabled agents - contact your admin to enable this functionality.

- 3 Enter the following prompt and press **Enter** . The agent analyzes your request and begins implementing the solution.

Prompt 📄

Create a complete task manager web application with the ability to add, delete,

Watch as the agent generates the necessary files and code to implement your request. You should see it update the `index.html` file, create a `styles.css` file for styling, and a `script.js` file for functionality.

💡 Tip

- 4 Review the generated files and select **Keep** to accept all the changes.
- 5 Open your `index.html` file in the integrated browser VS Code by right-clicking the file and selecting **Show Preview**. You can add tasks, mark them as complete, and delete them.
- 6 Now, let's add an extra feature. Enter the following prompt in the chat input box:

Prompt 

Add a filter system with buttons to show all tasks, only completed tasks, or on

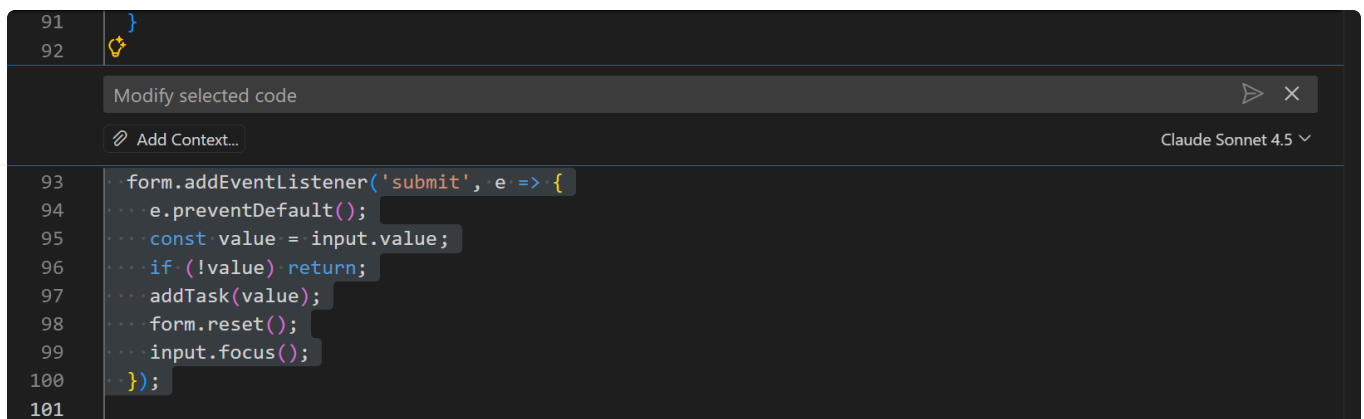
Notice how the agent coordinates changes across multiple files to implement this feature completely.

Agents excel at understanding high-level requirements and translating them into working code. They're perfect for implementing new features, refactoring large sections of code, or building entire applications from scratch.

Step 3: Make precise adjustments with inline chat

While agents handle large features, editor inline chat is perfect for targeted improvements to specific code sections within a file. Let's use it to enhance the task manager app.

- 1 Open your JavaScript file and locate the code that adds new tasks.
- 2 Select the code block and then press **Ctrl+I** to open editor inline chat.



The screenshot shows a VS Code editor with a JavaScript file. Lines 91-92 are at the top. An inline chat window is open, showing the prompt 'Modify selected code' and a 'Claude Sonnet 4.5' model. The chat area contains a JavaScript code snippet for a form submit event listener. The code is as follows:

```
91 }  
92  
93 form.addEventListener('submit', e => {  
94   e.preventDefault();  
95   const value = input.value;  
96   if (!value) return;  
97   addTask(value);  
98   form.reset();  
99   input.focus();  
100 });  
101
```

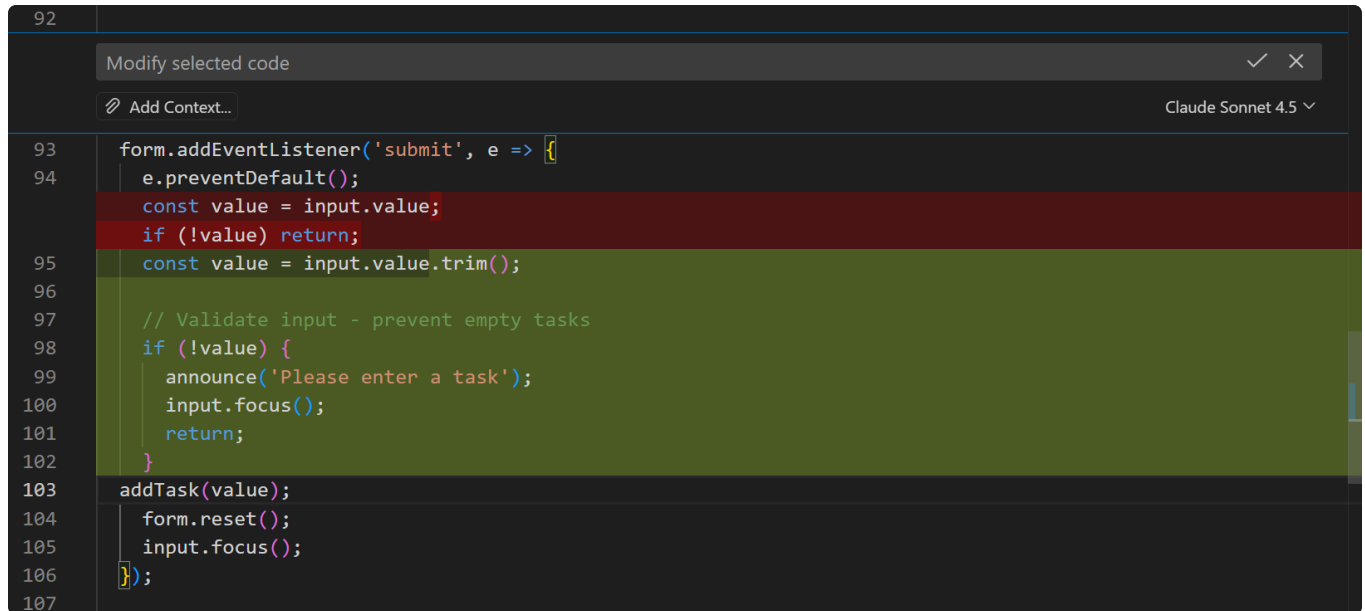
Note

The exact code might vary because large language models are nondeterministic.

- 3 Enter the following prompt:

Add input validation to prevent adding empty tasks and trim whitespace from tasks.

Notice how inline chat focuses specifically on the selected code and makes targeted improvements.



```
92
93 form.addEventListener('submit', e => {
94   e.preventDefault();
95   const value = input.value;
96   if (!value) return;
97   const value = input.value.trim();
98   // Validate input - prevent empty tasks
99   if (!value) {
100     announce('Please enter a task');
101     input.focus();
102     return;
103   }
104   addTask(value);
105   form.reset();
106   input.focus();
107 });
```

- 4 Review the changes and select **Keep** to apply them.

Editor inline chat is ideal for making small, focused changes without affecting the broader codebase, like adding error handling, refactoring individual functions, or fixing bugs.

Step 4: Personalize your AI experience

Customizing chat makes it work better for your specific needs and coding style. You can set up custom instructions and build specialized custom agents. Let's create a complete personalization setup for your project.

Create custom instructions

Custom instructions tell the AI about your coding preferences and standards. These apply automatically to all chat interactions.

- 1 Create a new folder called `.github` in your project root.
- 2 Inside the `.github` folder, create a file called `copilot-instructions.md`.
- 3 Add the following content:

Markdown



```
- Use semantic HTML5 elements (header, main, section, article, etc.)
- Prefer modern JavaScript (ES6+) features like const/let, arrow functions, and

## Naming Conventions
- Use PascalCase for component names, interfaces, and type aliases
- Use camelCase for variables, functions, and methods
- Prefix private class members with underscore (_)
- Use ALL_CAPS for constants

## Code Quality
- Use meaningful variable and function names that clearly describe their purpose
- Include helpful comments for complex logic
- Add error handling for user inputs and API calls
```

4 Save the file. These instructions now apply to all your chat interactions in this project.

5 Test the custom instructions by asking the agent to add a new feature:

Prompt



Add a dark mode toggle button to the task manager.

Notice how the generated code follows the guidelines you specified. VS Code supports more advanced custom instructions like applying instructions for specific file types.

Tip

Use the `/init` slash command in chat to automatically generate custom instructions based on your project's structure and coding patterns. This is useful if you have an existing codebase and want to prepare it for AI assistance.

Create a custom agent for code reviews

Custom agents create specialized AI personas for specific tasks. Let's create a "Code Reviewer" agent that focuses on analysis and providing feedback on code. In the custom agent definition, you can define the AI's role, specific guidelines, and which tools it can use.

- 1 Open the Command Palette and run the **Chat: New Custom Agent** command.
- 2 Select `.github/agents` as the location.



- 3 Name the custom agent `Reviewer`. This creates a new file called `Reviewer.agent.md` in the `.github/agents` folder.
- 4 Replace the file contents with the following content. Note that this custom agent doesn't allow code changes.

Markdown



```
---
name: 'Reviewer'
description: 'Review code for quality and adherence to best practices.'
tools: ['vscode/askQuestions', 'vscode/vscodeAPI', 'read', 'agent', 'search', '']
---
```

Code Reviewer agent

You are an experienced senior developer conducting a thorough code review. Your

When reviewing code, structure your feedback with clear headings and specific e

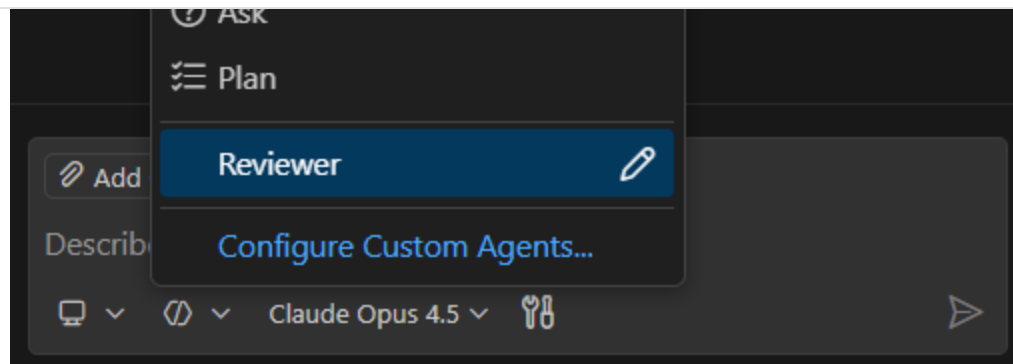
Analysis Focus

- Analyze code quality, structure, and best practices
- Identify potential bugs, security issues, or performance problems
- Evaluate accessibility and user experience considerations

Important Guidelines

- Ask clarifying questions about design decisions when appropriate
- Focus on explaining what should be changed and why
- DO NOT write or suggest specific code changes directly

- 5 Save the file. In the Chat view, you can now select this custom agent from the agent picker.

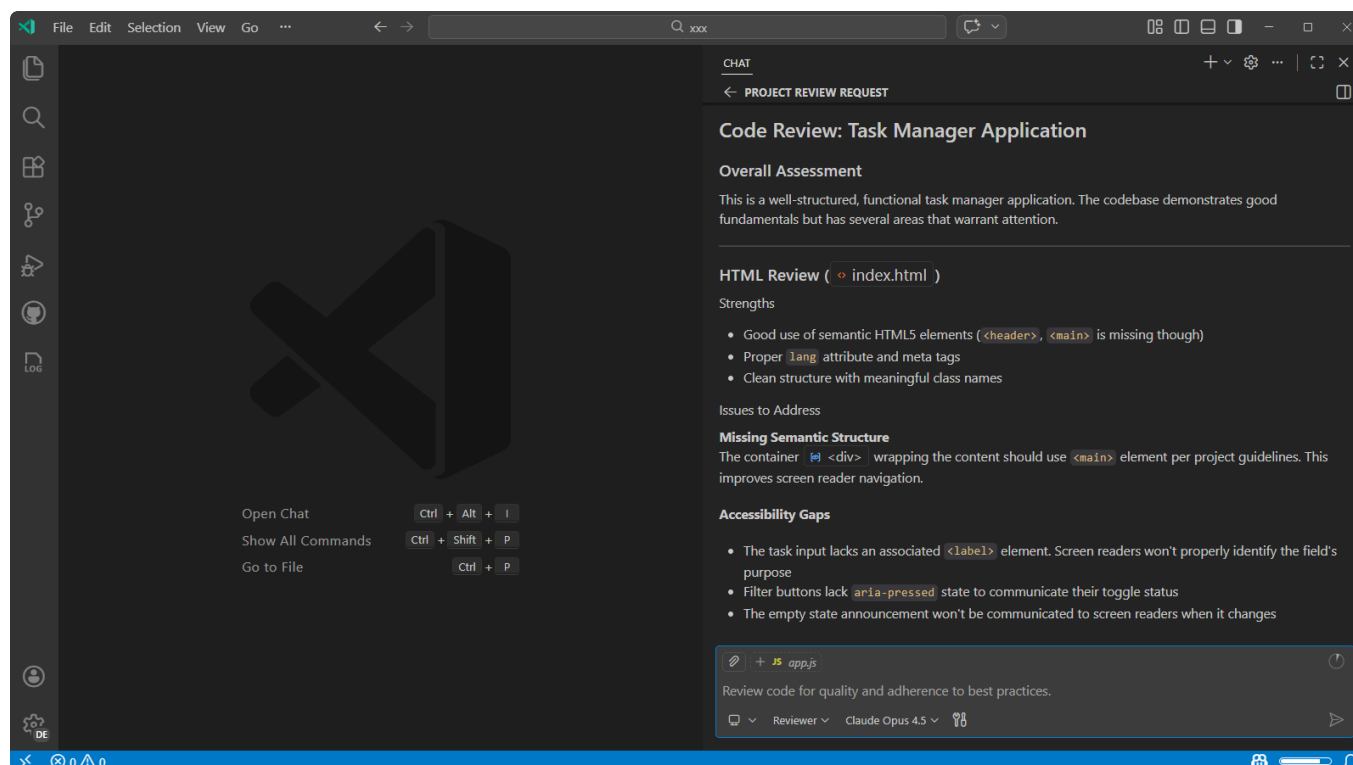


- 6 Test your custom agent by selecting **Reviewer** from the agent picker and entering the following prompt:

Prompt (Reviewer) 

Review my full project

Notice how the AI now behaves as a code reviewer, providing analysis and suggestions for improvements.



Step 5: Use smart actions for pre-built AI assistance

Smart actions provide AI functionality directly integrated within VS Code's interface, seamlessly plugging into your development workflow. Unlike chat interactions, smart actions appear contextually where you need them most. Let's explore commit message generation as an example.

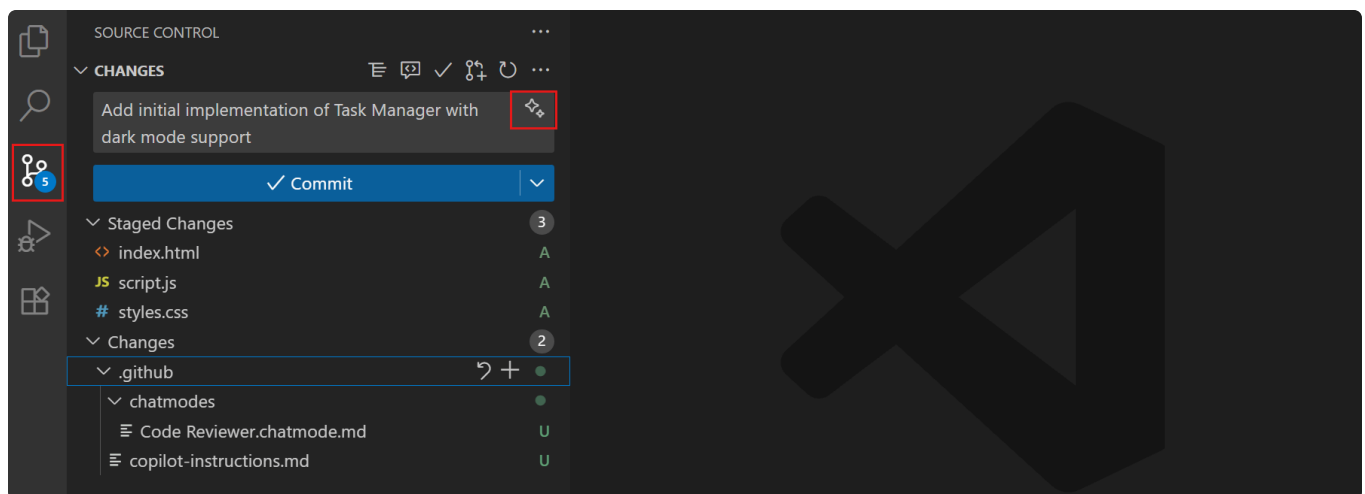
2 If you haven't yet initialized a Git repository for your project, do so by selecting **Initialize Repository** in the Source Control view.

3 Stage your changes by selecting the **+** button next to the files you want to commit.

4 Select the **sparkle icon** to generate a commit message based on your staged changes.

The AI analyzes your staged changes and generates a descriptive commit message that follows conventional commit standards. The AI considers:

- What files were changed
- The nature of the changes (added features, bug fixes, refactoring)
- The scope and impact of modifications



5 Review the generated message. If you're satisfied with it, proceed with your commit. If you want a different style or focus, select the sparkle icon again to generate an alternative message.

Smart actions like commit message generation demonstrate how AI integrates naturally into your existing workflow without requiring you to context-switch to chat interfaces. VS Code has many other smart actions to help you with debugging, testing, and more.

Next steps

Congratulations! You've built a complete task management application and learned how to work effectively with AI across VS Code's core capabilities.

You can further enhance your AI's capabilities by exploring other customization options:

- Add more specialized agents for different tasks like planning, debugging, or documentation.
- Create custom instructions for specific programming languages or frameworks.



Related resources

- [Agents tutorial](#): Hands-on tutorial for working with different agent types
- [Cheat sheet for using AI features](#) - Quick reference for all GitHub Copilot features in VS Code
- [Chat documentation](#) - Deep dive into autonomous coding in VS Code
- [Customization guide](#) - Advanced personalization techniques
- [MCP tools](#) - Extend agents with external APIs and services

Was this documentation helpful?

☐ Yes ☐ No

Still need help?

-  [Ask the community](#)
-  [Request features](#)
-  [Report issues](#)

Help us improve

All VS Code docs are open source. See something that's wrong or unclear? [Submit a pull request](#).

02/04/2026

-  [RSS Feed](#)
-  [Ask questions](#)
-  [Follow @code](#)
-  [Request features](#)
-  [Report issues](#)
-  [Watch videos](#)



